

Partition to Sequencing with Deadlines and Set-Up Times

Working manuscript for expert review
24 September 2026

Abstract

We give an explicit polynomial-time reduction from Partition to single-machine task sequencing with deadlines and compiler-dependent set-up times. Each input weight determines a compiler, an early task, and a late task. A long separator and two deadlines force the total weight of late tasks on either side of the separator to equal half the input sum. Every feasible target schedule therefore yields a Partition witness; infeasibility yields the prescribed no-solution output. The construction uses $2n + 2$ tasks and $n + 1$ compilers, with polynomial binary encoding size. Exhaustive and independently solved finite instances test the executable maps, while the proof covers all legal inputs and target outputs.

1. Introduction

Compiler changes consume time in a single-machine schedule. Deadline feasibility then depends on how tasks of the same compiler are batched. Garey and Johnson list this problem as SS6 (p. 238) and attribute its Partition reduction to Bruno and Downey [1], [2]. The corresponding hardness direction is known. A currently circulated rule sketch assigns input items to two fixed compiler classes and leaves the deadlines unspecified; its worked example even has unequal class sums [3]. We give a complete construction, a solution decoder, and a proof that covers every batching order.

The construction uses one compiler per input item. Each compiler has an early unit task that must precede a separator and a late task whose position can be chosen. An early position spends that item's processing weight before the separator; a late position pays its compiler setup again after the separator. The two capacity bounds force equality. This is a complete rule for the stated problem, without a claim that the particular gadget is new relative to Bruno and Downey's inaccessible full proof.

2. Problems and output semantics

A Partition instance is a nonempty sequence $a_0, \dots, a_{n-1}$ of positive binary-encoded integers. A valid output is a set of indices P with $\sum_{i \in P} a_i = \frac{\sum_i a_i}{2}$, or NO-SOLUTION exactly when no such set exists.

A target instance consists of tasks j with positive processing times p_j , positive deadlines d_j , and compiler labels c_j . Each compiler c has a nonnegative setup time s_c . Tasks are nonpreemptive on one machine. The first task has no initial setup charge. A later task incurs s_{c_j} exactly when its compiler differs from the preceding task's compiler. A valid output is a permutation meeting every deadline, or NO-SOLUTION exactly when no such permutation exists. Idle time can be deleted without worsening any deadline, so permutations with no idle time suffice.

3. Construction and recovery

Write $A = \sum_{i=0}^{n-1} a_i$, $B = \lfloor \frac{A}{2} \rfloor$, and define

$$T = 1 + n + A + B, \quad D = T + 1 + n + 2A + B. \quad (1)$$

There is a marker compiler with setup zero and, for each item i , a compiler c_i with setup a_i . Construct the following tasks in index order. The task names also identify their role in the proof.

Task	Processing	Deadline	Compiler
M (start)	1	1	marker
E_i (early)	1	T	c_i
L_i (late)	a_i	D	c_i
H (separator)	T	$2T$	marker

Table 1: The constructed tasks. Each row with index i denotes one task for every input item.

Task M has index 0; the pair E_i, L_i has indices $1 + 2i, 2 + 2i$; and H has index $2n + 1$. The output decoder finds H in a valid schedule and returns the indices whose L_i tasks precede it. It returns source NO-SOLUTION on target NO-SOLUTION. It reconstructs these indices from the source and target output in a fresh process.

The separator separates two charging regimes. The schematic in Table 2 shows where an item's late task may appear; its exact order within each side is unrestricted in the soundness proof.

M	all E_i ; selected L_i	H	remaining L_i
-----	----------------------------	-----	-----------------

Table 2: Schematic order in any feasible schedule. The early deadline forces every E_i before H ; the two bounds force the weight of L_i tasks on each side to be $\frac{A}{2}$. Tasks may have other orders and extra setups.

4. Correctness

Theorem. The construction maps every legal Partition instance to a legal target instance. For every valid target output y , the decoder returns a valid Partition output. Both maps are deterministic and polynomial-time.

We first bound every feasible schedule. Since all processing times are positive and $d_M = 1$, M must be first. Every E_i must precede H : otherwise the processing of M , H , and that E_i alone would make E_i finish after T . Let P index the L_i tasks before H , and put $w(P) = \sum_{i \in P} a_i$.

The deadline $d_H = 2T$ implies that work before H occupies at most T time units. Before H the schedule executes M , all E_i , and the L_i with $i \in P$. It also enters every item compiler from a different compiler at least once, because M is first. Thus

$$1 + n + A + w(P) \leq T = 1 + n + A + B, \quad \text{so} \quad w(P) \leq B. \quad (2)$$

The setup lower bound permits arbitrary extra visits and any batching order.

All task processing totals $1 + n + A + T$. The first entry into each item compiler costs A in total. For each item outside P , L_i follows H while E_i precedes it, so the schedule reenters compiler c_i after the marker compiler and pays at least another a_i . Every task deadline is at most D , hence the final completion obeys

$$T + 1 + n + A + A + (A - w(P)) \leq D = T + 1 + n + 2A + B, \quad \text{so} \quad A - w(P) \leq B. \quad (3)$$

If A is odd, Equation 2 and Equation 3 cannot both hold. If A is even, they force $w(P) = \frac{A}{2}$. Therefore the decoder returns a Partition witness from **every** feasible schedule, including schedules with additional switches.

For the converse, let P be a Partition witness. Schedule M , then one block E_i, L_i for each $i \in P$ and one block E_i for each item outside P , then H , then each remaining L_i . The blocks may appear in any item order. First entries cost A in setups. Before H the work is $1 + n + A + w(P) = T$, so all E_i meet T , and H finishes at $2T$. The remaining tasks have total processing $A - w(P) = B$ and total new setups B , so the schedule finishes at $2T + 2B = D$. Every L_i meets D . This proves target feasibility exactly when Partition is feasible. Consequently target NO-SOLUTION is decoded correctly as well.

The construction is legal because all task lengths and deadlines in Table 1 are positive, setup times are nonnegative, and every task has a defined compiler. This finishes the theorem.

5. Complexity and limits

There are $2n + 2$ tasks and $n + 1$ compilers. Since T and D are $O(n + A)$, each numerical field has $O(\log n + \log A)$ bits. The total target encoding has $O(n(\log n + \log A))$ bits, including compiler indices. Integer addition, serialization, and output recovery take polynomial bit time in the source and target-output lengths. The executable CLI removes Python’s default decimal-conversion digit cap, so its JSON representation admits arbitrary finite integer inputs.

The result gives an explicit F/G rule for a known hardness direction. It does not establish a new complexity classification or that this separator gadget differs from the original 1978 proof. The target oracles used for finite checks are exponential in the number of tasks; no practical target-solving speed claim follows.

References

- [1] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979. [Online]. Available: <https://perso.limos.fr/~palafour/PAPERS/PDF/Garey-Johnson79.pdf>
- [2] J. Bruno and P. Downey, “Complexity of Task Sequencing with Deadlines, Set-Up Times and Changeover Costs,” *SIAM Journal on Computing*, vol. 7, no. 4, pp. 393–404, 1978, doi: 10.1137/0207031.
- [3] CodingThrust, “[Rule] Partition to Sequencing with Deadlines and Set-Up Times.” [Online]. Available: <https://github.com/CodingThrust/problem-reductions/issues/474>

A. Verification and reproducibility

The implementation and checks are in `campaigns/partition-deadlines-setup-times/work/`. Python 3.12.14 and uv 0.12.17 were used; the locked dependency is `z3-solver==5.1.0.0`. The prepared checker passed 100 source cases and 128 actual target outputs, including 28 feasible and 72 no-solution targets. The separate brute-force verifier passed 86 source cases and 109 outputs, including two 4301-digit input cases. These are finite tests of the executable maps, separate from the proof and independent review.

From the repository root, run:

```
uv sync --locked
uv run python campaigns/partition-deadlines-setup-times/work/check.py --self-test
uv run python campaigns/partition-deadlines-setup-times/work/check.py --candidate
    campaigns/partition-deadlines-setup-times/work/algorithm.py
uv run python campaigns/partition-deadlines-setup-times/work/verify.py --candidate
    campaigns/partition-deadlines-setup-times/work/algorithm.py
```

For one forward call, supply a JSON object of the form `{"numbers":[1,2,3]}` on standard input to `algorithm.py`. For recovery, run `algorithm.py --extract` with `{"source":{"numbers":[1,2,3]}, "target_solution":[...]}`; the target solution must be a valid schedule for the constructed instance, or `"NO-SOLUTION"` when no such schedule exists. Candidate revision `b18ecdd` passed focused independent re-review. Source corpus sizes are one to eight items; the separate exhaustive verifier covers item sequences of length at most three over values one to four, plus the two large-integer cases. No larger solver scale has been established.